ACADEMIC SUBMISSION
from:
LEARNWELL INSTITUTE

PROJECT REPORT
ON

TWEET MANAGEMENT SYSTEM

A TWITTER-STYLE LINKED LIST ENGINE







STUDENT NAMES	:	Tarun Manghani / Bharat Soni
COURSE	:	BCA
SEMESTER	:	2nd Semester
SUBJECT	:	C++ Programming & Data Structure
SESSION	:	2025-26
COLLEGE	:	Dezyne École College

Today A Reader,
Tomorrow A Leader

Dezyne École College

www.dezyneecole.com

Title of Project

TWEET MANAGEMENT SYSTEM

Subject Name

Data Structure and C++

Submitted By

TARUN MANGHANI & BHARAT SONI&

Department - BCA

Semester - Second

Session – 2025-26

Project Report of

Tweet Management System

Submitted to

Dezyne École College

Towards the partial

Fulfilment of **BCA 2nd Semester**

By

Tarun Manghani

&

Bharat Soni

Today A Reader,
Tomorrow A Leader

www.dezyneecole.com

2025-2026

Acknowledgement

We take this opportunity to express our sincere gratitude to Dezyne École College for providing us with the knowledge, resources, and guidance essential to the successful completion of this project.

Our deepest thanks to our respected Principal, Dr. Vinita Mathur, for her visionary leadership and continuous encouragement. Her commitment to academic excellence and student growth has been a constant source of motivation and pride.

We are equally thankful to all our faculty members for their consistent support, expert advice, and valuable insights that shaped our learning and helped us achieve this milestone.

Lastly, we extend heartfelt thanks to everyone who directly or indirectly contributed to the success of this project.

We are proud to be part of Dezyne École College, an institution that truly nurtures professionalism, innovation, and purpose.

With sincere appreciation,
TARUN MANGHANI & BHARAT SONI
Department of BCA
Dezyne École College, Ajmer

INDEX

Introduction 1

Singly vs Doubly — At a Glance 1

Why Not an Array? 2

Singly Linked List — All Operations 3

Node Structure & Memory Representation 3

Insert at Head — O(1) 4

Insert at Tail — O(1) with tail pointer 5

Insert at Position k — O(k) 6

Delete Operations — 3 Cases 7

Traversal — O(n) 9

Search by Keyword — O(n) 10

Count Nodes — O(1) Best Practice 11

Reverse the List — O(n) Time, O(1) Space 12

Doubly Linked List — Insertion, Deletion & Traversal 13

Insert at Head — O(1) 14

Insert at Tail — O(1) 15

Insert at Position k — O(k) 16

Delete Head — O(1) 17

Delete Tail — O(1) ★ Key Advantage Over Singly 18

Delete Middle Node — O(1) with pointer / O(n) if search required 19

Forward & Backward Traversal — O(n) each 20

Complete Complexity Comparison & Exam Guide 22

Full Complexity Table 22

When to Use Which? 23

Key Exam Questions & Answers 24

Common Mistakes to Avoid in Exams 24

Summary — The Complete Picture 26

Linklist CODE :- 27

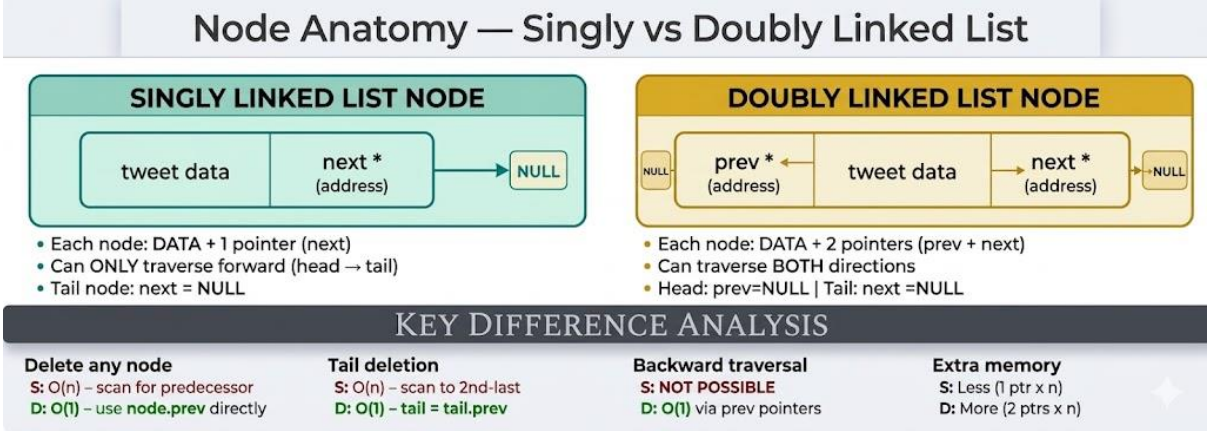
Doublyll program 35

Main Project linklist 43

Introduction

Elaborate Linked list ?

A Linked List is a linear data structure where elements (called nodes) are NOT stored in contiguous memory locations. Each node holds its data and one or two pointers — memory addresses — that link it to neighbouring nodes. The nodes can live anywhere in RAM; the pointers stitch them into a logical sequence.



Real-World Context — Twitter Timeline

Twitter's home timeline shows tweets in reverse-chronological order — newest at the top, oldest at the bottom. Three operations dominate this system:

- **Insert at head:** A new tweet always goes at the very top. This must be instant — O(1).
- **Delete from any position:** Content moderation removes any tweet immediately — O(1).
- **Traverse from head to tail:** Displaying the timeline scrolls from newest to oldest — O(n).

Singly vs Doubly — At a Glance

PROPERTY	SINGLY LINKED LIST	DOUBLY LINKED LIST
Pointers per node	1 (NEXT Only)	2 (PREV + NEXT)
Traversal direction	Forward Only (Head → Tail)	Both Directions
Delete tail	O(N) — Must Scan To 2nd-Last	O(1) — Use Tail.Prev

Delete any node (with ptr)	O(N) — Scan For Predecessor	O(1) — Both Neighbours Known
Extra memory	Less (1 Ptr × N)	More (2 Ptr × N = +8 Bytes/Node)
Typical use	Stacks, Hash Chaining, Simple Queues	Timelines, Browsers, LRU Cache, Deques

Why Not an Array?

OPERATION	ARRAY	LINKED LIST	TWITTER'S NEED
Insert At Front	O(N) — Shift All Elements Right	O(1) — Rewire 2 Pointers	New Tweet On Top — Must Be Instant
Delete Any Element	O(N) — Shift Remaining Elements	O(1) — Rewire Neighbours	Remove Any Tweet Instantly
Access By Index	O(1) — Direct Formula	O(N) — Traverse From Head	Not Needed For Sequential Display
Dynamic Size (Grow)	Needs Reallocation	Trivial — Just Add A Node	Tweet Count Is Unpredictable
Search	O(N) Linear, O(Log N) Binary	O(N) Linear Only	Both Require Traversal For Unsorted Data

Singly Linked List — All Operations

Node Structure & Memory Representation

In a singly linked list each node stores data and ONE pointer — next — which holds the memory address of the following node. The last node's next = NULL. We maintain a head pointer to always find the start, and optionally a tail pointer for efficient end-access.

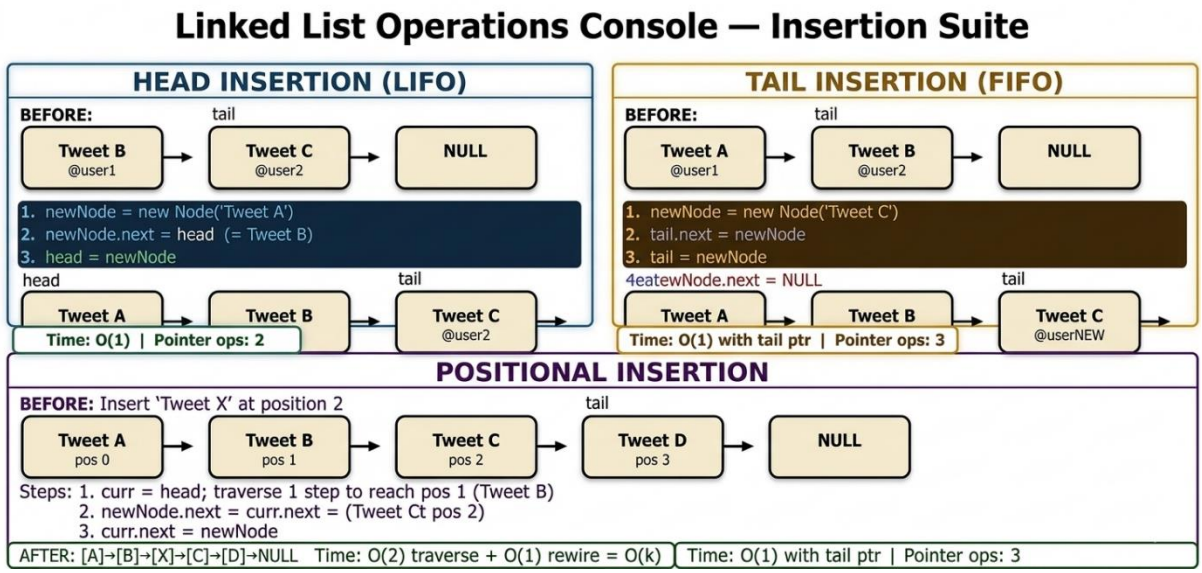
C++ struct:

Memory layout example:

```
head → [Tweet4 | 0x3A10] → [Tweet3 | 0x1F40] → [Tweet2 | 0x7C80] → [Tweet1 | NULL]
       at 0x2B00          at 0x3A10           at 0x1F40           at 0x7C80
       ↑ newest                                                   ↑ oldest (tail)

Nodes are SCATTERED in RAM — not consecutive like an array.
The only way to reach Tweet2 is: start at head, follow next, follow next, arrive.
```

Insert at Head — O(1)



Concept

The newest tweet always goes at the top of the timeline. Head insertion achieves this in exactly 2 pointer operations — always O(1) regardless of how many tweets already exist.

Algorithm are followed :-

1. **Create new node:** Tweet Node* new Node = new Tweet Node(text, user); — allocates heap memory, sets next = NULL.
2. **Connect forward:** new Node->next = head; — new node now points to the current head (will become 2nd node).
3. **Move head pointer:** head = new Node; — head now refers to the new node.
4. **Handle empty list:** If the list was empty, also set tail = new Node;
5. **Increment counter:** size++;

```
void insertAtHead(string text, string user) {
TweetNode* newNode = new TweetNode(text, user);

if (head == nullptr) {        // empty list
head = tail = newNode;
} else {
newNode->next = head;      // Step 2: link forward
head = newNode;           // Step 3: advance head
}
size++;
}
```

```
// Example trace:
// Before: head → [B] → [C] → NULL
// newNode.next = B   (newNode now points to B)
// head = newNode
// After:  head → [A] → [B] → [C] → NULL   Time: O(1)
```

Why O(1)? No loop, no traversal. The same 2 pointer operations run whether the list has 3 nodes or 3 million nodes.

Insert at Tail — O(1) with tail pointer

Without tail pointer: Must traverse all n nodes to reach the last one, then append. Cost = O(n).

With tail pointer (always recommended): Directly access tail, attach new node, advance tail. Cost = O(1).

Algorithm are followed (with tail pointer) :-

1. **Create new node:** TweetNode* newNode = new TweetNode(text, user); — newNode->next = NULL.
2. **Handle empty list:** If head == NULL: head = tail = newNode; return.
3. **Connect:** tail->next = newNode; — current tail now points to the new node.
4. **Advance tail:** tail = newNode; — tail pointer moves to the new last node.
5. **Increment counter:** size++;

```
void insertAtTail(string text, string user) {
TweetNode* newNode = new TweetNode(text, user);

if (head == nullptr) {
head = tail = newNode;
} else {
tail->next = newNode;      // current tail points forward
tail = newNode;            // tail advances to new node
}
size++;
}

// Before: head→[A]→[B]→NULL  tail=B
// tail.next = newNode (= C)
// tail = C
// After:  head→[A]→[B]→[C]→NULL  tail=C   Time: O(1)
```

Insert at Position k — O(k)

Three sub-cases

- **k = 0:** Same as insertAtHead() — O(1).
- **k = size (last position + 1):** Same as insertAtTail() — O(1).

0 < k < size (middle): Traverse k-1 steps to find the predecessor node, then rewire 2 pointers.

Algorithm for Middle Insert :-

6. Create Newnode.
7. Start Curr = Head; Traverse K-1 Steps: For(Int I=0; I<K-1; I++) Curr = Curr->Next;
8. Newnode->Next = Curr->Next; — New Node Points To What Curr Was Pointing To.
9. Curr->Next = Newnode; — Curr Now Points To New Node.
10. Size++;

```
void insertAtPosition(int k, string text, string user) {
    if (k <= 0)     { insertAtHead(text, user); return; }
    if (k >= size)  { insertAtTail(text, user); return; }

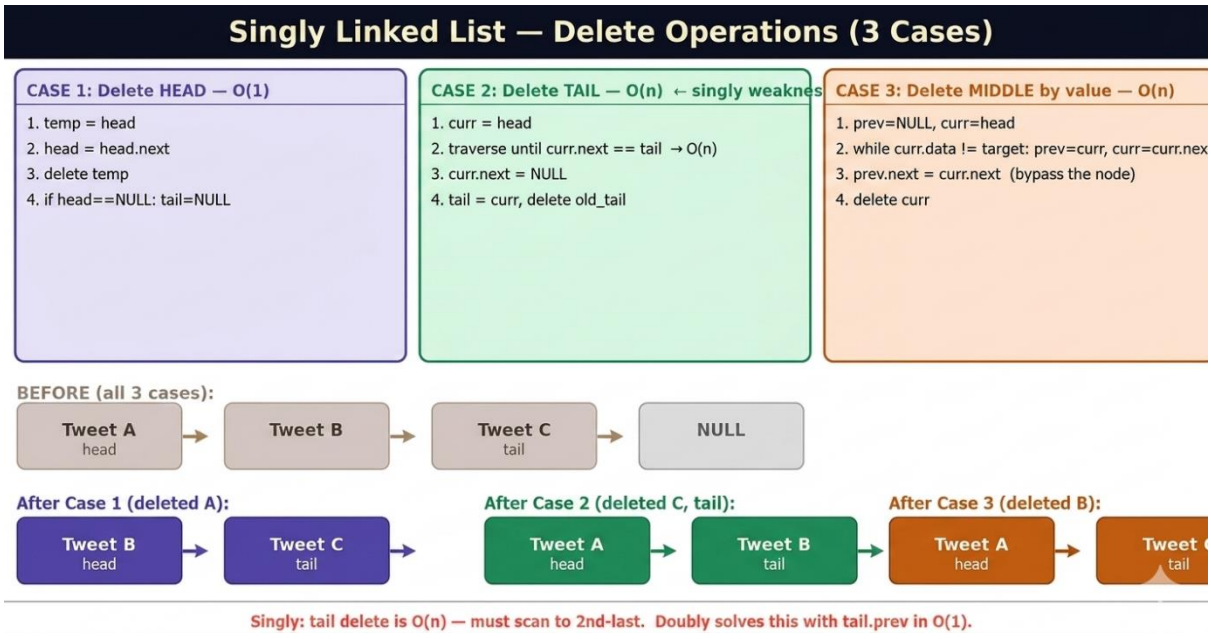
    TweetNode* newNode = new TweetNode(text, user);
    TweetNode* curr    = head;

    for (int i = 0; i < k-1; i++)   // traverse k-1 steps
        curr = curr->next;

    newNode->next = curr->next;   // new node points past curr
    curr->next    = newNode;      // curr points to new node
    size++;
}

// Insert 'X' at pos 2 in [A]→[B]→[C]→[D]:
// curr = B (after 1 step, since k-1=1)
// X.next = C   ;   B.next = X
// Result: [A]→[B]→[X]→[C]→[D]   Time: O(2) = O(k)
```

Delete Operations — 3 Cases



Case 1 — Delete Head — O(1)

Moving the head pointer one step forward effectively removes the first node. No traversal needed.

11. `temp = head;` — save the node to free later.
12. `head = head->next;` — head now points to the 2nd node (new head).
13. If head != NULL: it was a single-node list: set `tail = NULL;`
14. `delete temp;` — free heap memory.
15. `size--;`

Case 2 — Delete Tail — O(n) ← Singly's Weakness

Problem: To delete the tail we must update the second-to-last node's next pointer to NULL. In a singly linked list the only way to find that node is to traverse from head — O(n). This is why doubly linked lists are preferred when tail deletion is frequent.

16. Start `curr = head;` traverse until `curr->next == tail;`
17. `curr->next = NULL;` — second-to-last now terminates the list.
18. `delete tail;` then `tail = curr;`
19. `size--;`

Case 3 — Delete Middle Node by Value — O(n)

Find the target node and its predecessor simultaneously using two pointers, then bypass the target.

20. `prev = NULL; curr = head;`
21. Traverse: while curr != NULL and curr->data != target: `prev = curr; curr = curr->next;`
22. If curr == NULL: value not found — return.
23. `prev->next = curr->next;` — predecessor skips over the target node.

24. If curr == tail: tail = prev; — update tail pointer.
25. delete curr; size--;

```
void deleteByValue(string target) {
if (!head) { cout << "Empty list"; return; }

// Special case: deleting head
if (head->tweetText == target) { deleteHead(); return; }

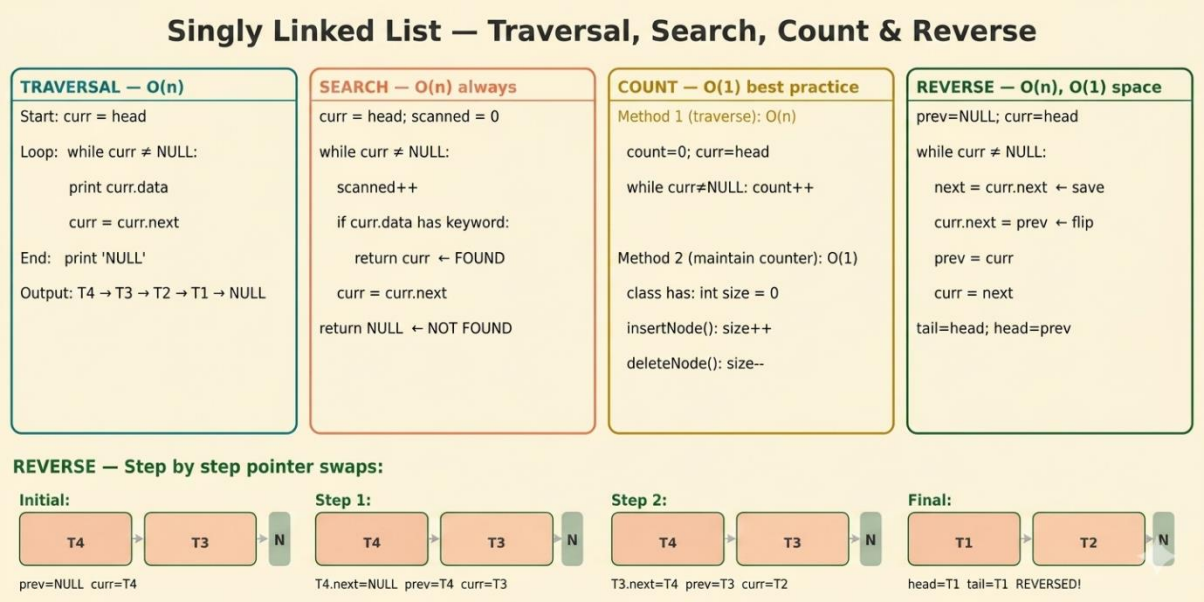
TweetNode* prev = nullptr;
TweetNode* curr = head;

while (curr != nullptr && curr->tweetText != target) {
prev = curr;
curr = curr->next;
}

if (curr == nullptr) { cout << "Not found"; return; }

prev->next = curr->next;        // bypass the node
if (curr == tail) tail = prev;   // update tail if needed
delete curr;
size--;
}
```

Traversal — O(n)



Concept

Traversal visits every node exactly once, following next pointers from head until NULL. This is how Twitter renders your timeline from top to bottom — each tweet displayed in order.

26. **curr = head; — start at newest tweet.**
27. **Loop: while (curr != nullptr) — stop when we fall off the end.**
28. **Inside loop: process curr->data; then curr = curr->next;**

```
void displayTimeline() {
TweetNode* curr = head;
int position    = 1;

while (curr != nullptr) {
cout << position << ". [@" << curr->username << "] "
<< curr->tweetText
<< "  | Likes: " << curr->likes << endl;
curr = curr->next;     // follow next pointer
position++;
}
cout << "NULL  (end of timeline)" << endl;
}
// Time: O(n)  |  Space: O(1) — only one curr pointer variable
```

Critical exam point: Always check curr != nullptr BEFORE accessing curr->data. Accessing a NULL pointer causes a segmentation fault (crash). This is the #1 beginner error in linked list programs.

Search by Keyword — O(n)

Why O(n) only: Linked lists have no index formula. To find a node you must start at head and examine each node one by one. Best case O(1) if found at head. Worst case O(n) if at tail or absent.

Why binary search is impossible: Binary search requires jumping to the middle element. In a linked list, reaching node i requires following i pointers from head — O(i). So finding the middle costs O(n/2), making binary search O(n log n) — worse than linear O(n).

```
TweetNode* searchByKeyword(string keyword) {
                TweetNode* curr    = head;
                    int       scanned  = 0;

                while (curr != nullptr) {
                        scanned++;
            if (curr->tweetText.find(keyword) != string::npos) {
        cout << "Found after " << scanned << " nodes" << endl;
            return curr;      // found — return pointer to node
                        }
                    curr = curr->next;
                        }
    cout << "Not found — scanned all " << scanned << " nodes (O(n))" << endl;
                    return nullptr;
                        }
```

Count Nodes — O(1) Best Practice

Approach	Algorithm	Time	Recommendation
Traverse and count	curr=head; while curr≠NULL: count++; curr=curr.next	O(n) per call	Avoid for frequent calls
Maintain size counter	size++ on insert, size-- on delete, getCount() returns size	O(1) per call	Always use this approach

```
// Maintain a 'size' member variable in the class
int getCount() { return size; }  // O(1) — just return the variable

// Increment whenever you insert
void insertAtHead(string t, string u) { /* ... */; size++; }

// Decrement whenever you delete
void deleteHead() { /* ... */; size--; }
```

Reverse the List — O(n) Time, O(1) Space

Concept

Reversing a singly linked list flips all next pointers so the old tail becomes the new head. The key is maintaining three pointers — prev, curr, next_node — so we never lose our place.

No extra list: The reversal happens in-place by redirecting existing pointers. Space = O(1) — a major advantage over creating a new reversed copy.

Three-Pointer Algorithm

29. **Initialise:** prev = nullptr; curr = head;
30. Loop: while (curr != nullptr)
31. Save next: TweetNode* nextNode = curr->next;
32. Flip pointer: curr->next = prev; ← this is the reversal
33. Advance: prev = curr; curr = nextNode;
34. **After loop:** tail = head; then head = prev; ← prev is now the old tail = new head

```
void reverseTimeline() {
TweetNode* prev     = nullptr;
 TweetNode* curr     = head;
TweetNode* nextNode = nullptr;

tail = head;          // old head will become new tail

while (curr != nullptr) {
nextNode    = curr->next;   // save — we're about to overwrite it
curr->next  = prev;         // FLIP: point backward
prev        = curr;         // advance prev
curr        = nextNode;     // advance curr
}
head = prev;   // prev ended at the old tail = new head
}

// Trace for [T4]→[T3]→[T2]→[T1]→NULL:
// Iter 1: T4.next = NULL    prev=T4  curr=T3
// Iter 2: T3.next = T4      prev=T3  curr=T2
// Iter 3: T2.next = T3      prev=T2  curr=T1
// Iter 4: T1.next = T2      prev=T1  curr=NULL  (stop)
// head = T1  →  [T1]→[T2]→[T3]→[T4]→NULL   Time: O(n), Space: O(1)
```

Doubly Linked List — Insertion, Deletion & Traversal

What the prev Pointer Adds

A doubly linked list adds one extra pointer — prev — to every node. This single addition costs 8 bytes per node on a 64-bit system but unlocks three major advantages:

- **O(1) delete of any node — both neighbours are immediately accessible via prev and next. No scanning needed.**
- **O(1) tail deletion — tail.prev gives the second-to-last node directly. No traversal.**
- **Backward traversal — start at tail, follow prev pointers. Impossible in singly linked lists.**

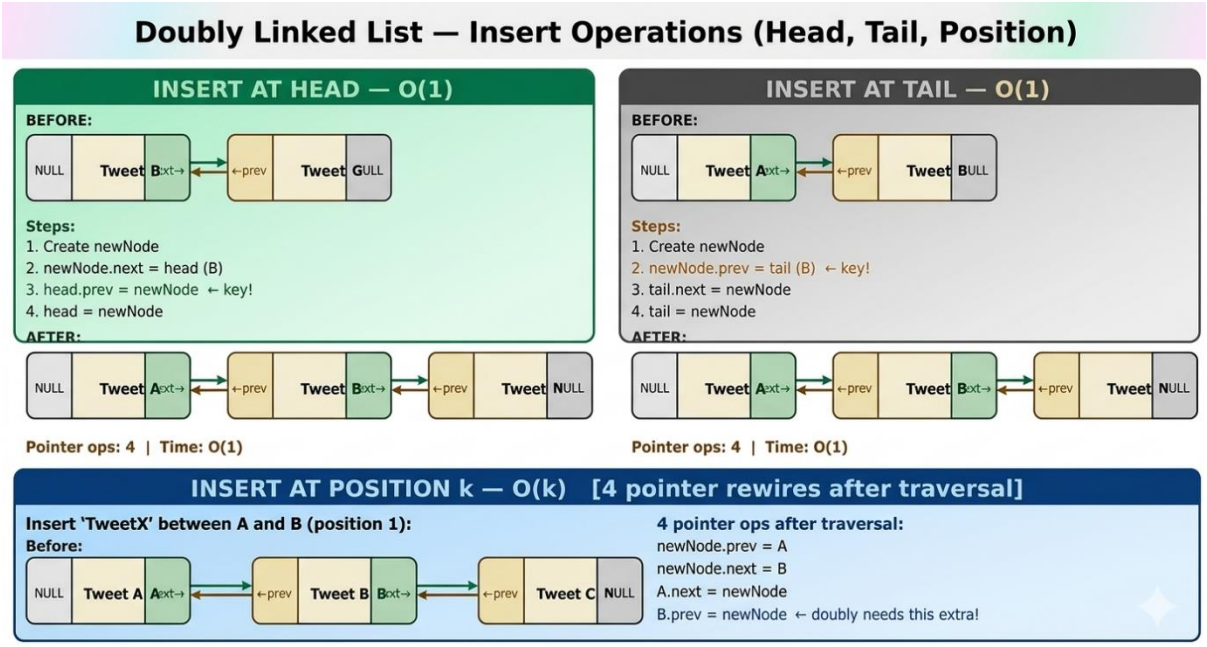
C++ Node Structure for Doubly Linked List

```
struct TweetNodeDL {
    string      tweetText;
    string      username;
    int         likes;
    TweetNodeDL* prev;   // ← NEW: points to previous (newer) tweet
    TweetNodeDL* next;   // points to next (older) tweet

    TweetNodeDL(string text, string user) {
        tweetText = text; username = user; likes = 0;
        prev = nullptr;  // NULL initially
        next = nullptr;
    }
};

// Memory layout:
// NULL ← [T4 | prev←→next] ↔ [T3 | prev←→next] ↔ [T2 | prev←→next] ↔ [T1] → NULL
//          ↑ head                                                    ↑ tail
//   head.prev = NULL  |  tail.next = NULL
```

Insert at Head — O(1)



Extra Step vs Singly

Compared to singly insertion at head, doubly adds one step: setting the old head's prev pointer back to the new node.

Algorithm — 4 pointer operations

35. **Create:** TweetNodeDL* newNode = new TweetNodeDL(text, user); — prev=NULL, next=NULL.
36. If empty: head = tail = newNode; return.
37. **Connect forward:** newNode->next = head; — new node points to old head.
38. **Connect backward:** head->prev = newNode; — old head points BACK to new node. ← extra step vs singly!
39. **Advance head:** head = newNode;
40. size++;

```
void insertAtHead(string text, string user) {
TweetNodeDL* newNode = new TweetNodeDL(text, user);

if (!head) { head = tail = newNode; size++; return; }

newNode->next = head;      // 1. new node → old head
head->prev    = newNode;   // 2. old head ← new node  (DOUBLY key step)
head          = newNode;   // 3. advance head
// newNode->prev stays NULL — it is the new head
size++;
}
```

Insert at Tail — O(1)

Always O(1): Doubly linked lists always maintain a tail pointer. Appending is direct — no traversal required in either the empty or non-empty case.

Algorithm — 4 pointer operations

41. **Create:** TweetNodeDL* newNode = new TweetNodeDL(text, user);
42. If empty: head = tail = newNode; return.
43. **Connect backward:** newNode->prev = tail; — new node looks BACK at current tail.
44. **Connect forward:** tail->next = newNode; — current tail points forward to new node.
45. **Advance tail:** tail = newNode;
46. size++;

```
void insertAtTail(string text, string user) {
TweetNodeDL* newNode = new TweetNodeDL(text, user);

if (!tail) { head = tail = newNode; size++; return; }

newNode->prev = tail;      // 1. new node ← current tail
tail->next    = newNode;   // 2. current tail → new node
tail          = newNode;   // 3. tail advances
// newNode->next stays NULL — it is the new tail
size++;
}
```

Insert at Position k — O(k)

Middle case — 4 pointer rewires (vs 2 in singly)

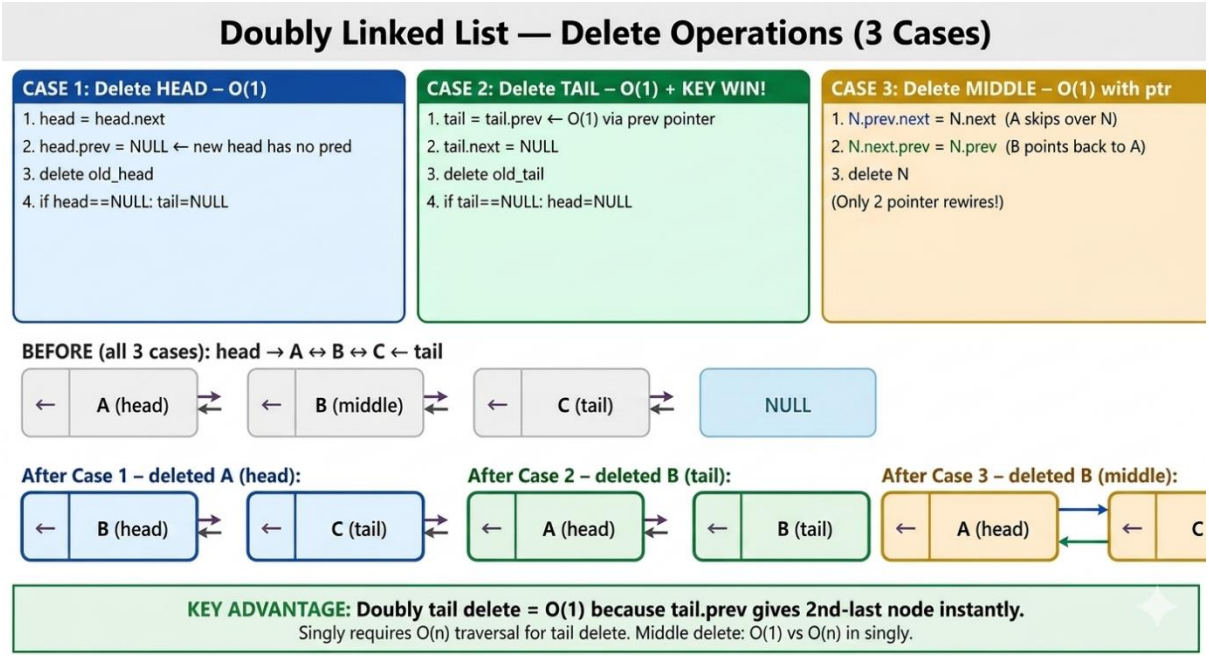
Traverse to position k-1 (same as singly), but now must wire FOUR pointers: both neighbours' prev and next must be updated.

```
// Insert between node A (at pos k-1) and node B (at pos k)
TweetNodeDL* A = /* traverse to pos k-1 */;
TweetNodeDL* B = A->next;

newNode->prev = A;      // new node looks back to A
newNode->next = B;      // new node looks forward to B
A->next       = newNode; // A now points to new node
if (B) B->prev = newNode; // B now points back to new node
else   tail = newNode;    // inserting at end: update tail

// Result: NULL ← [A] ↔ [NEW] ↔ [B] → NULL
// Time: O(k) traverse + O(1) rewire  =  O(k)
```

Delete Head — O(1)



Algorithm

47. **Check:** If head == NULL: empty list, return.
48. **Save:** TweetNodeDL* temp = head;
49. **Advance head:** head = head->next;
50. If head != NULL: head->prev = nullptr; — new head has no predecessor.
51. Else (list now empty): tail = nullptr;
52. **Free:** delete temp; size--;

Delete Tail — O(1) ★ Key Advantage Over Singly

Why This is Doubly's Key Advantage

In a singly linked list, deleting the tail requires O(n) traversal to find the second-to-last node (which must have its next set to NULL). In a doubly linked list, tail.prev ALREADY points to the second-to-last node — O(1), no scanning!

Algorithm — 3 pointer operations

53. **Check:** If tail == NULL: empty list, return.
54. **Save:** TweetNodeDL* temp = tail;
55. **Advance tail BACKWARD:** tail = tail->prev; ← ONLY possible because of the prev pointer!
56. If tail != NULL: tail->next = nullptr; — new tail has no successor.
57. Else (list now empty): head = nullptr;
58. delete temp; size--;

```
void deleteTail() {
  if (!tail) return;

  TweetNodeDL* temp = tail;
tail = tail->prev;          // O(1) — prev pointer does the work

  if (tail) tail->next = nullptr; // new tail's next = NULL
    else     head = nullptr;      // list became empty

  delete temp;
    size--;
     }
// Singly equivalent would need:  while(curr->next != tail) curr=curr->next;  → O(n)
```

Delete Middle Node — O(1) with pointer / O(n) if search required

The Biggest Advantage

If you already have a direct pointer to the node to delete, O(1) is achievable because both neighbours are accessible via prev and next. Just 2 rewires bypass the node completely.

Algorithm — 2 pointer rewires

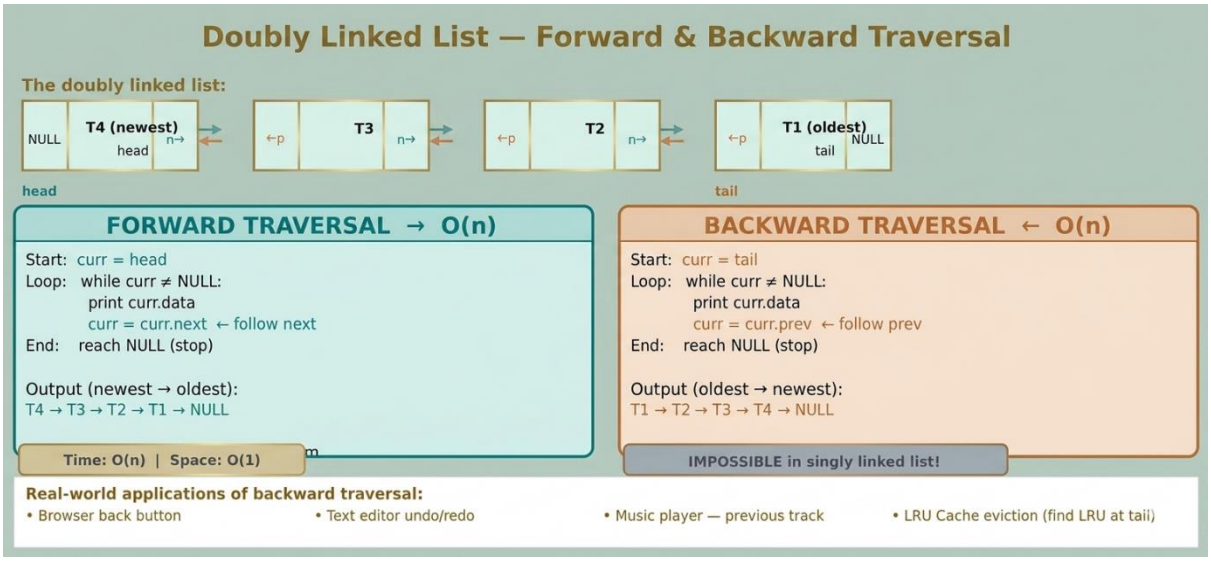
59. Check edge cases: if N == head → call deleteHead(); if N == tail → call deleteTail().
60. **A.next = N.next:** N->prev->next = N->next; — predecessor skips over N.
61. **B.prev = N.prev:** N->next->prev = N->prev; — successor points back past N.
62. delete N; size--;

```
void deleteNode(TweetNodeDL* N) {
if (!N) return;
if (N == head) { deleteHead(); return; }
if (N == tail) { deleteTail(); return; }

// Middle node — both neighbours already known!
N->prev->next = N->next;   // A skips over N → points to B
N->next->prev = N->prev;   // B points back past N → points to A
delete N;
size--;
// Only 2 pointer rewires — O(1) — NO SCANNING NEEDED
}

// Singly equivalent would need O(n) scan to find N's predecessor.
```

Forward & Backward Traversal — O(n) each



Forward Traversal (Newest → Oldest)

Identical to singly traversal. Start at head, follow next pointers. This is how Twitter renders your timeline top-to-bottom.

```
void displayForward() {
TweetNodeDL* curr = head;        // start at newest tweet
while (curr != nullptr) {
cout << "[@" << curr->username << "] " << curr->tweetText << endl;
curr = curr->next;
}
cout << "NULL" << endl;
}
// Output: T4 → T3 → T2 → T1 → NULL   (newest first)
```

Backward Traversal (Oldest → Newest) ← Impossible in Singly

Unique to doubly linked lists. Start at tail, follow prev pointers. This direction is physically impossible in a singly linked list because there is no prev pointer.

```
void displayBackward() {
TweetNodeDL* curr = tail;        // start at OLDEST tweet
while (curr != nullptr) {
cout << "[@" << curr->username << "] " << curr->tweetText << endl;
curr = curr->prev;          // follow PREV pointer
}
cout << "NULL" << endl;
}
// Output: T1 → T2 → T3 → T4 → NULL   (oldest first)
// Use case: 'Show oldest tweets first' view on Twitter
```

Real-World Applications of Backward Traversal

Application	How Backward Traversal is Used
Browser back button	Navigate history in reverse (most recent → oldest visited)
Text editor undo/redo	Move backward through change history to undo actions
Music player Previous Track	Go backward through the playlist
LRU Cache eviction	Tail is LRU item — accessed via backward from end, evicted O(1)
Twitter / Social Media	Show thread in chronological order (oldest reply first)

Complete Complexity Comparison & Exam Guide

Complete Complexity Comparison — Singly vs Doubly Linked List

Operation	Singly — Time	Doubly — Time	Space	Key Insight
Insert at head	O(1)	O(1)	O(1)	Always 2 pointer ops
Insert at tail	O(n)*	O(1)	O(1)	*O(1) if tail ptr maintained
Insert at pos k	O(k)	O(k)	O(1)	Traverse k steps; k≤n
Delete head	O(1)	O(1)	O(1)	Move head to head.next
Delete tail	O(n)	O(1) ★	O(1)	★ Doubly uses tail.prev directly
Delete (pointer)	O(n)	O(1) ★	O(1)	★ KEY ADVANTAGE of doubly
Delete (search)	O(n)	O(n)	O(1)	Must scan first in both
Forward traversal	O(n)	O(n)	O(1)	Visit every node once
Backward traversal	N/A ✗	O(n)	O(1)	✗ Impossible in singly
Search	O(n)	O(n)	O(1)	No random access in either
Count (counter)	O(1)	O(1)	O(1)	Maintain size variable
Reverse	O(n)	O(n)	O(1)	Swap prev/next for all n nodes

★ = Doubly linked list advantage | ✗ = Feature not available | Use tail pointer to make tail inserts O(1) in singly

Full Complexity Table

Operation	Singly Time	Doubly Time	Space	Notes
Insert at head	O(1)	O(1)	O(1)	Singly: 2 ops | Doubly: 4 ops
Insert at tail	O(n) / O(1)*	O(1)	O(1)	*O(1) with tail pointer maintained
Insert at position k	O(k)	O(k)	O(1)	Traverse k steps; doubly needs 4 rewires
Delete head	O(1)	O(1)	O(1)	Move head forward
Delete tail	O(n)	O(1) ★	O(1)	★ DOUBLY KEY ADVANTAGE: use tail.prev
Delete with pointer	O(n)	O(1) ★	O(1)	★ Doubly: both neighbours known instantly
Delete by value	O(n)	O(n)	O(1)	Must search first; unlink is O(1)
Forward traversal	O(n)	O(n)	O(1)	Visit every node once
Backward traversal	N/A — impossible	O(n)	O(1)	Only doubly can go backwards

Search	O(n)	O(n)	O(1)	Binary search impossible — no random access
Count (counter var)	O(1)	O(1)	O(1)	Maintain size variable — always do this
Reverse	O(n)	O(n)	O(1)	Swap next/prev for all n nodes in-place

When to Use Which?

Use Case	Best Choice	Reason
Twitter timeline, feed	Doubly	O(1) delete anywhere, both-direction traversal
Browser back/forward	Doubly	Traverse both directions, O(1) add/remove ends
Undo/Redo system	Doubly	Walk backward and forward through actions
LRU Cache	Doubly + HashMap	O(1) delete any node when cache is full
Simple stack (LIFO)	Singly	Only need push/pop at head, less memory
Hash table chaining	Singly	Simple forward-only chain, less overhead
Print job queue (FIFO)	Singly + tail ptr	Enqueue at tail O(1), dequeue at head O(1)
Playlist with prev/next	Doubly	Song navigation in both directions

Key Exam Questions & Answers

Q1: Why is inserting at head O(1) in a linked list but O(n) in an array?

A: An array stores elements consecutively in memory. Inserting at index 0 requires shifting ALL existing elements one position right — that is n copy operations. A linked list has no such constraint — we just create a new node and rewire 2 pointers (newNode.next = head, head = newNode). These 2 operations are the same whether the list has 3 nodes or 3 million.

Q2: Why is singly tail deletion O(n) but doubly tail deletion O(1)?

A: To delete the tail we must update the second-to-last node's next pointer to NULL. In a singly linked list the only way to find that node is to traverse from head — O(n) steps. In a doubly linked list, tail.prev ALREADY holds the address of the second-to-last node. We access it in O(1) with no scanning needed.

Q3: Why is binary search impossible on a linked list?

A: Binary search requires O(1) access to the middle element (arr[mid]). In a linked list there is no index formula — finding node i requires following i pointers from head, costing O(i). So finding the middle costs O(n/2), making binary search O(n log n) in total — worse than just scanning linearly at O(n). Always use linear search on linked lists.

Q4: What is the space complexity of reversing a linked list?

A: O(1) — the three-pointer algorithm (prev, curr, nextNode) reverses the list in-place by redirecting existing pointers. No extra list or array is created. All n pointer rewires happen within the same memory nodes that already exist.

Q5: A node N is to be deleted from a doubly linked list. How many pointer operations are needed if you already have a pointer to N?

A: Exactly 2 operations: (1) N.prev.next = N.next — the predecessor skips over N. (2) N.next.prev = N.prev — the successor points back past N. Both neighbours are immediately accessible through N's own pointers, so no scanning is needed. Time = O(1).

Common Mistakes to Avoid in Exams

- **Mistake 1 — NULL pointer access:** Accessing curr->data without first checking curr != NULL. Always check for NULL before dereferencing any pointer.
- **Mistake 2 — Forgetting tail pointer:** Writing insert at tail as O(n) when a tail pointer makes it O(1). Always assume the class maintains both head and tail.
- **Mistake 3 — Missing prev update in doubly:** When inserting between A and B, forgetting B.prev = newNode. All 4 pointers must be wired.
- **Mistake 4 — Not updating tail on delete:** If you delete the current tail node (middle delete of last node), forgetting to update the tail pointer leaves a dangling pointer.
- **Mistake 5 — Memory leak:** Every node created with new must eventually be freed with delete. Forgetting delete in the destructor causes memory leaks.
- **Mistake 6 — Claiming O(log n) search:** Linked lists cannot do binary search. Search is always O(n). Never write O(log n) for linked list search.

Summary — The Complete Picture

Assignment A6 — Quick Reference

Twitter use case: Doubly linked list. New tweets insert at head in O(1). Any tweet deletes in O(1). Timeline scrolls forward via next, backward via prev.

Singly operations: Insert head O(1), insert tail O(1)*, insert pos O(k), delete head O(1), delete tail O(n), delete middle O(n), traversal O(n), search O(n), count O(1)*, reverse O(n).

Doubly operations: Insert head O(1), insert tail O(1), insert pos O(k), delete head O(1), delete tail O(1)★, delete middle O(1)★ with ptr, forward O(n), backward O(n).

★ marks doubly's advantages. The price: 8 extra bytes per node for the prev pointer.

The fundamental trade-off: Arrays give O(1) random access but O(n) insert/delete. Linked lists give O(1) insert/delete at known positions but O(n) access. Choose based on which operation is most frequent in your application.

Linklist CODE :-

```
#include<iostream>
#include<conio.h>
#include<string>
using namespace std;
class NODE
{
public:
string ID;
string UNAME;
string TWEET;
NODE *NEXT;
NODE(string x, string y, string z)
{
ID = x;
UNAME = y;
TWEET = z;
NEXT = nullptr;
}
};
class LinkList
{
public:
string TID;
int TCNT;
string UName, UTweet;
NODE *START,*END;
int countNode;
LinkList(){
TCNT = 1001;
countNode = 0;
START = nullptr;
END = nullptr;
}
void getData()
{
TID = "TWEET"+ to_string(TCNT);
cout<<"\nEnter TID -- "<<TID<<endl;
//getline(cin,TID);
cout<<"Enter User Name -- ";
//cin.ignore();
getline(cin,UName);
cout<<"Enter Tweet -- ";
getline(cin,UTweet);
}
char continueCheck()
{
char c;
cout<<"\n\nAre you.. \n";
cout<<"1. Continue with Same\n";
```

```
50. cout<<"2. Go to Main Previous Menu\n";
51. cout<<"3. Go to Main Menu..\n";
52. cout<<"4. Exit\n";
53. c = getch();
54. if ( c == '4'){
55. thanksLine();
56. exit(0);
57. }else{
58. return (c);
59. }
60. }
61. int insertNode(int opr)
62. {
63. system("cls");
64. if ( opr == 1){
65. cout<<"\n\nInsert At Beginning . \n\n";
66. getData();
67. NODE *TEMP = new NODE(TID,UName, UTweet);
68. if ( TEMP == nullptr){
69. cout<<"Insufficient Memory.. \n";
70. return 0;
71. }
72. if ( START == nullptr){
73. END = START = TEMP;
74. }
75. else{
76. TEMP->NEXT = START;
77. START = TEMP;
78. }
79. countNode++;
80. TCNT++;
81. }
82. else if ( opr == 2){
83. cout<<"\n\nInsert At End . \n\n";
84. getData();
85. NODE *TEMP = new NODE(TID,UName, UTweet);
86. if ( TEMP == nullptr){
87. cout<<"Insufficient Memory.. \n";
88. return 0;
89. }
90. if ( START == nullptr){
91. END = START = TEMP;
92. }
93. else{
94. END->NEXT = TEMP;
95. END = TEMP;
96. }
97. countNode++;
98. TCNT++;
99. }
100. else if ( opr == 3){
```

```
101.            cout<<"\n\nInsert At Given Position . \n\n";
102.            cout<<"Total Nodes -- "<<countNode<<"\n\n";
103.             int pos;
104.            getData();
105.            cout<<"Enter Position Between 0 to "<<countNode<<" -- ";
106.            cin>>pos;
107.
108.            NODE *TEMP = new NODE(TID,UName, UTweet);
109.            if ( TEMP == nullptr){
110.            cout<<"Insufficient Memory.. \n";
111.            return 0;
112.            }
113.            int i;
114.            NODE TEMP1 = START;
115.            for ( i = 2 ; i < pos ; i++){
116.            TEMP1 = TEMP1->NEXT;
117.            }
118.            if ( START == nullptr){
119.            END = START = TEMP;
120.            }
121.            else{
122.            TEMP->NEXT = TEMP1->NEXT;
123.            TEMP1->NEXT = TEMP;
124.            if ( TEMP1 == END){
125.            END = TEMP1->NEXT;
126.            }
127.            }
128.            countNode++;
129.            TCNT++;
130.            }
131.            char c = continueCheck();
132.            if (c == '1')
133.            insertNode(opr);
134.            else if ( c == '2')
135.            insertionOperation();
136.            else if ( c == '3')
137.            MainMenu();
138.            }
139.            void showNode()
140.            {
141.            cout<<"\n\nLink List -- \n\n";
142.            cout<<" Head -> ";
143.            NODE *TEMP = START;
144.            if ( START == nullptr){
145.            cout<<"null \n\nlist is empty\n";
146.            }
147.            else{
148.            while ( TEMP!= nullptr){
149.            cout<<"["<<TEMP->ID<<", ";
150.            cout<<TEMP->UNAME<<", ";
151.            cout<<TEMP->TWEET<<"] -> ";
```

```
152.            TEMP = TEMP->NEXT;
153.            }
154.          cout<<"Null";
155.            }
156.            }
157.          void countNodes()
158.            {
159.          system("cls");
160.          cout<<"Count Nodes Module --\n\n";
161.          cout<<"Total Number of Nodes -- "<<countNode<<" Nodes Available\n\n"
162.            ; }
163.           int insertionOperation()
164.            {
165.          system("cls");
166.          cout<<"\nInsertion Operation..\n\n";
167.           cout<<"1. Insert At Beginning\n";
168.           cout<<"2. Insert At End\n";
169.           cout<<"3. Insert After Data\n";
170.           cout<<"4. Go to Main Menu\n";
171.           cout<<"5. Exit.\n";
172.            char c = getch();
173.             if ( c == '1'){
174.            insertNode(1);
175.            }
176.           else if ( c == '2'){
177.            insertNode(2);
178.            }
179.           else if ( c == '3'){
180.            insertNode(3);
181.            }
182.           else if ( c == '4'){
183.             MainMenu();
184.            }
185.           else if ( c == '5'){
186.             thanksLine();
187.           exit(0);
188.            }
189.            }
190.
191.           int deleteNode(int opr)
192.            {
193.           NODE *TEMP;
194.            if ( opr == 1){
195.          cout<<"\n\nDelete From Beginning . \n\n";
196.           if ( START == nullptr)
197.            {
198.          cout<<"LinkList is Empty.. \n\n";
199.           return 0;
200.            }
201.          else if ( START->NEXT == nullptr){
202.           TEMP = START;
```

```
203.            END = START = nullptr;
204.            }
205.            else{
206.            TEMP = START;
207.            START = START->NEXT;
208.            }
209.            countNode--;
210.            delete(TEMP);
211.            cout<<"\nNode Deleted Successfully\n\n";
212.            }
213.            else if ( opr == 2){
214.            cout<<"\n\nDelete From End . \n\n";
215.            if ( START == nullptr)
216.            {
217.            cout<<"LinkList is Empty.. \n\n";
218.            return 0;
219.            }
220.            else if ( START->NEXT == nullptr){
221.            TEMP = START;
222.            END = START = nullptr;
223.            }
224.            else{
225.            TEMP = START;
226.            int c = 1;
227.            while ( TEMP->NEXT != END)
228.            {
229.            cout<<c++<<endl;
230.            TEMP = TEMP->NEXT;
231.            }
232.            NODE *T1 = END;
233.            TEMP->NEXT = nullptr;
234.            END = TEMP;
235.            cout<<END->UNAME<<endl;
236.            delete(T1);
237.            }
238.            countNode--;
239.            cout<<"\nNode Deleted Successfully\n\n";
240.            }
241.            else if ( opr == 3){
242.            cout<<"\n\nDelete From Given Position . \n\n";
243.
244.            if ( START == nullptr)
245.            {
246.            cout<<"LinkList is Empty.. \n\n";
247.            return 0;
248.            }
249.            int pos;
250.            cout<<"Enter Position for Delete (1 - "<<countNode<<") - ";
251.            cin>>pos;
252.            if ( pos == 1 && START == END)
253.            {
```

```
254.            TEMP = START;
255.          START = END = nullptr;
256.            }
257.          else{
258.            TEMP = START;
259.          for (int i = 2 ; i < pos ; i++)
260.            {
261.          TEMP = TEMP->NEXT;
262.            }
263.          if (TEMP->NEXT == END){
264.            NODE *T1 = END;
265.          TEMP->NEXT = nullptr;
266.            END = TEMP;
267.            }
268.          else{
269.          NODE *T1 = TEMP->NEXT;
270.          TEMP->NEXT = T1->NEXT;
271.            delete(T1);
272.            }
273.            }
274.            countNode--;
275.        cout<<"\nNode Deleted Successfully\n\n";
276.            }
277.          char c = continueCheck();
278.            if (c == '1')
279.          deleteNode(opr);
280.            else if ( c == '2')
281.          deleteOperation();
282.            else if ( c == '3')
283.            MainMenu();
284.            }
285.          int deleteOperation()
286.            {
287.            system("cls");
288.          cout<<"\nDeletion Operation..\n\n";
289.          cout<<"1. Delete From Beginning\n";
290.          cout<<"2. Delete From End\n";
291.          cout<<"3. Delete From Given Position\n";
292.          cout<<"4. Go to Main Menu\n";
293.          cout<<"5. Exit.\n";
294.          char c = getch();
295.            if ( c == '1'){
296.          deleteNode(1);
297.            }
298.          else if ( c == '2'){
299.          deleteNode(2);
300.            }
301.          else if ( c == '3'){
302.          deleteNode(3);
303.            }
304.          else if ( c == '4'){
```

```
            MainMenu();
        }
    else if ( c == '5'){
        thanksLine();
        exit(0);
        }
        }
    /search module start/

int searchTweetInNode()
        {
  cout<<"\n\nSearch Tweet in Nodes -- \n\n";
        cout<<"* Head -> ";
    NODE *TEMP = START;
    if ( START == nullptr){
  cout<<"null \n\nlist is empty\n";
        return 0;
        }
    else{
  cout<<"\n\nEnter User Name To Find Tweets -- ";
        string uname;
    int counter = 0;
    getline(cin,uname);
    while ( TEMP!= nullptr){
    if (TEMP->UNAME == uname)
        {
  cout<<"\n["<<TEMP->ID<<", ";
    cout<<TEMP->UNAME<<", ";
    cout<<TEMP->TWEET<<"]\n";
        counter++;
        }
    TEMP = TEMP->NEXT;
        }
    if ( counter == 0){
  cout<<"\nNo Tweet Found..\n\n";
        }
        }
    char c;
  cout<<"\n\nAre you .. \n";
  cout<<"1. Continue with Same\n";
  cout<<"2. Go to Main Menu..\n";
  cout<<"3. Exit\n";
    c = getch();
    if (c == '1')
  searchTweetInNode();
    else if ( c == '2')
    MainMenu();
    else if ( c == '3'){
    thanksLine();
    exit(0);
        }
```

```
356.            }
357.
358.          /Search Module End/
359.            int MainMenu()
360.            {
361.              system("cls");
362.    cout<<"========== LinkList Operation ==========\n\n";
363.              cout<<"1. Insertion Operation\n";
364.              cout<<"2. Traversal Operation\n";
365.               cout<<"3. Delete Operation\n";
366.               cout<<"4. Search Operation\n";
367.                cout<<"5. Count Nodes\n";
368.              cout<<"6. Go to Home Menu\n";
369.               cout<<"7. Exit\n";
370.              cout<<"Choose One of them -- ";
371.               char c = getch();
372.                if ( c == '1'){
373.              insertionOperation();
374.              }
375.              else if ( c == '2'){
376.                showNode();
377.              }
378.              else if ( c == '3'){
379.               deleteOperation();
380.              }
381.              else if ( c == '4'){
382.              searchTweetInNode();
383.              }
384.              else if ( c == '5'){
385.               countNodes();
386.              }
387.              else if ( c == '6'){
388.            MainLinkList::MainLinkListModule();
389.              }
390.              else if ( c == '7'){
391.               thanksLine();
392.                exit(0);
393.              }
394.    cout<<"\n\nDo you want to continue .. press 1 else any key..";
395.               c = getch();
396.               if ( c == '1'){
397.               MainMenu();
398.              }
399.              }
400.            void thanksLine()
401.            {
402.    cout<<"\n\nThanks for Using My APP\n\n";
403.            }
404.            };
```

Doublyll program

```
#include<iostream>
#include<conio.h>
#include<string>
using namespace std;
class NODED
{
public:
NODED *PREV;
string ID;
string UNAME;
string TWEET;
NODED *NEXT;
NODED(string x, string y, string z)
{
PREV = nullptr;
ID = x;
UNAME = y;
TWEET = z;
NEXT = nullptr;
}
};
class DLinkList
{
public:
string TID;
int TCNT;
string UName, UTweet;
NODED *START,*END;
int countNode;
DLinkList(){
TCNT = 1001;
countNode = 0;
START = nullptr;
END = nullptr;
}
void getData()
{
TID = "TWEET"+ to_string(TCNT);
cout<<"\nEnter TID -- "<<TID<<endl;
//getline(cin,TID);
cout<<"Enter User Name -- ";
//cin.ignore();
getline(cin,UName);
cout<<"Enter Tweet -- ";
getline(cin,UTweet);
}
char continueCheck()
{
char c;
cout<<"\n\nAre you.. \n";
```

```
51. cout<<"1. Continue with Same\n";
52. cout<<"2. Go to Main Previous Menu\n";
53. cout<<"3. Go to Main Menu..\n";
54. cout<<"4. Exit\n";
55. c = getch();
56. if ( c == '4'){
57. thanksLine();
58. exit(0);
59. }else{
60. return (c);
61. }
62. }
63. int insertNode(int opr)
64. {
65. system("cls");
66. if ( opr == 1){
67. cout<<"\n\nInsert At Beginning . \n\n";
68. getData();
69. NODED *TEMP = new NODED(TID,UName, UTweet);
70. if ( TEMP == nullptr){
71. cout<<"Insufficient Memory.. \n";
72. return 0;
73. }
74. if ( START == nullptr){
75. END = START = TEMP;
76. }
77. else{
78. TEMP->NEXT = START;
79. START->PREV = TEMP;
80. START = TEMP;
81. }
82. cout<<END->UNAME;
83. countNode++;
84. TCNT++;
85. }
86. else if ( opr == 2){
87. cout<<"\n\nInsert At End . \n\n";
88. getData();
89. NODED *TEMP = new NODED(TID,UName, UTweet);
90. if ( TEMP == nullptr){
91. cout<<"Insufficient Memory.. \n";
92. return 0;
93. }
94. if ( START == nullptr){
95. END = START = TEMP;
96. }
97. else{
98. END->NEXT = TEMP;
99. TEMP->PREV = END;
100. END = TEMP;
101. }
```

```
                countNode++;
                TCNT++;
                }
            else if ( opr == 3){
        cout<<"\n\nInsert At Given Position . \n\n";
        cout<<"Total Nodes -- "<<countNode<<"\n\n";
                int pos;
                getData();
    cout<<"Enter Position Between 0 to "<<countNode<<" -- ";
                cin>>pos;

            NODED *TEMP = new NODED(TID,UName, UTweet);
                if ( TEMP == nullptr){
            cout<<"Insufficient Memory.. \n";
                return 0;
                }
                int i;
            NODED *TEMP1 = START;
             for ( i = 2 ; i < pos ; i++){
             TEMP1 = TEMP1->NEXT;
                }
              if ( START == nullptr){
              END = START = TEMP;
                }
              else{

          TEMP->NEXT = TEMP1->NEXT;
            TEMP1->NEXT = TEMP;
            TEMP->PREV = TEMP1;
          TEMP->NEXT->PREV=TEMP;
             if ( TEMP1 == END){
            END = TEMP1->NEXT;
                }
                }
            countNode++;
                TCNT++;
                }
            char c = continueCheck();
              if (c == '1')
           insertNode(opr);
             else if ( c == '2')
          insertionOperation();
             else if ( c == '3')
              MainMenu();
                }
            int showNode()
                {
        cout<<"\n\nDoubly Link List Traversal -- \n\n";
             cout<<"* Head -> ";
            if ( START == nullptr){
        cout<<"null \n\nlist is empty\n";
```

```
153.            return 0;
154.            }
155.        cout<<"\n\n1. Forward Traversal..\n";
156.        cout<<"2. Backward Traversal\n";
157.        cout<<"3. Goto to Main Menu..\n";
158.        cout<<"4. Exit\n";
159.        char c = getch();
160.        if ( c == '1'){
161.        cout<<"\n\nForward Traversal\n\n";
162.        cout<<"null <- ";
163.        NODED *TEMP = START;
164.        while ( TEMP!= nullptr){
165.        cout<<"["<<TEMP->ID<<", ";
166.        cout<<TEMP->UNAME<<", ";
167.        cout<<TEMP->TWEET<<"] -> ";
168.        TEMP = TEMP->NEXT;
169.        }
170.        cout<<"Null\n\n";
171.        }
172.        else if ( c == '2'){
173.        cout<<"\n\nBackward Traversal\n\n";
174.        cout<<"null <- ";
175.        NODED *TEMP = END;
176.        while ( TEMP!= nullptr){
177.        cout<<"["<<TEMP->ID<<", ";
178.        cout<<TEMP->UNAME<<", ";
179.        cout<<TEMP->TWEET<<"] -> ";
180.        TEMP = TEMP->PREV;
181.        }
182.        cout<<"Null\n\n";
183.        }
184.        else if ( c == '3'){
185.        MainMenu();
186.        }
187.        else if (c == '4'){
188.        exit(0);
189.        }
190.        }
191.        void countNodes()
192.        {
193.        system("cls");
194.        cout<<"Count Nodes Module --\n\n";
195.        cout<<"Total Number of Nodes --
196.        "<<countNode<<" Nodes Available\n\n"
197.        ; }
198.        int insertionOperation()
199.        {
200.        system("cls");
201.        cout<<"\nInsertion Operation..\n\n";
202.        cout<<"1. Insert At Beginning\n";
203.        cout<<"2. Insert At End\n";
```

```
204.            cout<<"3. Insert After Data\n";
205.           cout<<"4. Go to Main Menu\n";
206.            cout<<"5. Exit.\n";
207.             char c = getch();
208.              if ( c == '1'){
209.             insertNode(1);
210.              }
211.             else if ( c == '2'){
212.             insertNode(2);
213.              }
214.             else if ( c == '3'){
215.             insertNode(3);
216.              }
217.             else if ( c == '4'){
218.              MainMenu();
219.              }
220.             else if ( c == '5'){
221.              thanksLine();
222.             exit(0);
223.              }
224.              }
225.
226.            int deleteNode(int opr)
227.              {
228.             NODED *TEMP;
229.              if ( opr == 1){
230.           cout<<"\n\nDelete From Beginning . \n\n";
231.             if ( START == nullptr)
232.              {
233.           cout<<"LinkList is Empty.. \n\n";
234.              return 0;
235.              }
236.           else if ( START->NEXT == nullptr){
237.              TEMP = START;
238.            END = START = nullptr;
239.              }
240.              else{
241.              TEMP = START;
242.            START = START->NEXT;
243.            START->PREV=nullptr;
244.              }
245.             countNode--;
246.             delete(TEMP);
247.           cout<<"\nNode Deleted Successfully\n\n";
248.              }
249.             else if ( opr == 2){
250.           cout<<"\n\nDelete From End . \n\n";
251.             if ( START == nullptr)
252.              {
253.           cout<<"LinkList is Empty.. \n\n";
254.              return 0;
```

```
255.            }
256.        else if ( START->NEXT == nullptr){
257.            TEMP = START;
258.          END = START = nullptr;
259.            }
260.            else{
261.          NODED *TEMP = END;
262.            END = END->PREV;
263.          END->NEXT = nullptr;
264.            delete(TEMP);
265.            }
266.            countNode--;
267.        cout<<"\nNode Deleted Successfully\n\n";
268.            }
269.            else if ( opr == 3){
270.        cout<<"\n\nDelete From Given Position . \n\n";
271.
272.            if ( START == nullptr)
273.            {
274.            cout<<"LinkList is Empty.. \n\n";
275.            return 0;
276.            }
277.            int pos;
278.        cout<<"Enter Position for Delete (1 - "<<countNode<<") - ";
279.            cin>>pos;
280.            if ( pos == 1 && START == END)
281.            {
282.            TEMP = START;
283.            START = END = nullptr;
284.            }
285.            else{
286.            TEMP = START;
287.            for (int i = 1 ; i < pos ; i++)
288.            {
289.            TEMP = TEMP->NEXT;
290.            }
291.            if (TEMP == END){
292.            TEMP->PREV->NEXT = nullptr;
293.            END = TEMP->PREV;
294.            delete(TEMP);
295.            }
296.            else{
297.            TEMP->PREV->NEXT = TEMP->NEXT;
298.            TEMP->NEXT->PREV = TEMP->PREV;
299.            delete(TEMP);
300.            }
301.            }
302.            countNode--;
303.        cout<<"\nNode Deleted Successfully\n\n";
304.            }
305.            char c = continueCheck();
```

```
306.            if (c == '1')
307.            deleteNode(opr);
308.            else if ( c == '2')
309.            deleteOperation();
310.            else if ( c == '3')
311.            MainMenu();
312.            }
313.            int deleteOperation()
314.            {
315.            system("cls");
316.            cout<<"\nDeletion Operation..\n\n";
317.            cout<<"1. Delete From Beginning\n";
318.            cout<<"2. Delete From End\n";
319.            cout<<"3. Delete From Given Position\n";
320.            cout<<"4. Go to Main Menu\n";
321.            cout<<"5. Exit.\n";
322.            char c = getch();
323.            if ( c == '1'){
324.            deleteNode(1);
325.            }
326.            else if ( c == '2'){
327.            deleteNode(2);
328.            }
329.            else if ( c == '3'){
330.            deleteNode(3);
331.            }
332.            else if ( c == '4'){
333.            MainMenu();
334.            }
335.            else if ( c == '5'){
336.            thanksLine();
337.            exit(0);
338.            }
339.            }
340.            /search module start/
341.
342.            int searchTweetInNode()
343.            {
344.            cout<<"\n\nSearch Tweet in Nodes -- \n\n";
345.            cout<<"* Head -> ";
346.            NODED *TEMP = START;
347.            if ( START == nullptr){
348.            cout<<"null \n\nlist is empty\n";
349.            return 0;
350.            }
351.            else{
352.            cout<<"\n\nEnter User Name To Find Tweets -- ";
353.            string uname;
354.            int counter = 0;
355.            getline(cin,uname);
356.            while ( TEMP!= nullptr){
```

```
            if (TEMP->UNAME == uname)
                {
            cout<<"\n["<<TEMP->ID<<", ";
             cout<<TEMP->UNAME<<", ";
            cout<<TEMP->TWEET<<"]\n";
                counter++;
                }
          TEMP = TEMP->NEXT;
                }
           if ( counter == 0){
        cout<<"\nNo Tweet Found..\n\n";
                }
                }
              char c;
         cout<<"\n\nAre you .. \n";
        cout<<"1. Continue with Same\n";
         cout<<"2. Go to Main Menu..\n";
              cout<<"3. Exit\n";
               c = getch();
               if (c == '1')
            searchTweetInNode();
             else if ( c == '2')
              MainMenu();
             else if ( c == '3'){
               thanksLine();
                exit(0);
                }
                }

          /Search Module End/
             int MainMenu()
                {
              system("cls");
    cout<<"========== LinkList Operation ==========\n\n";
        cout<<"1. Insertion Operation\n";
        cout<<"2. Traversal Operation\n";
         cout<<"3. Delete Operation\n";
         cout<<"4. Search Operation\n";
          cout<<"5. Count Nodes\n";
           cout<<"6. Exit\n";
        cout<<"Choose One of them -- ";
             char c = getch();
              if ( c == '1'){
           insertionOperation();
                }
             else if ( c == '2'){
               showNode();
                }
             else if ( c == '3'){
            deleteOperation();
                }
```

```
408.            else if ( c == '4'){
409.             searchTweetInNode();
410.            }
411.            else if ( c == '5'){
412.             countNodes();
413.            }
414.            else if ( c == '6'){
415.             thanksLine();
416.             exit(0);
417.            }
418.        cout<<"\n\nDo you want to continue .. press 1 else any key..";
419.             c = getch();
420.             if ( c == '1'){
421.             MainMenu();
422.            }
423.            }
424.          void thanksLine()
425.            {
426.        cout<<"\n\nThanks for Using My APP\n\n";
427.            }
428.            };
429.            /*
430.           int main()
431.            {
432.            DLinkList l;
433.           l.MainMenu();
434.        cout<<"\n\nThanks for Using My APP\n\n";
435.            }*/
```

Main Project linklist

```
1.  #include<iostream>
2.  #include<conio.h>
3.  #include"LinkList.cpp"
4.  #include"doublyll.cpp"
5.  class MainLinkList
6.  {
7.  public:
8.  LinkList l;
9.  DLinkList dl;
10. void MainLinkListModule()
11. {
12. system("cls");
13. cout<<"------ Welcome In Link List Project ------\n\n";
14. cout<<"1. Singly Link List \n\n";
15. cout<<"2. Doubly Link List \n\n";
16. cout<<"3. Exit \n\n";
17. cout<<"Choose any one of them....";
18. char c = getch();
19. if ( c == '1'){
```

```
20. l.MainMenu();
21. }
22. else if ( c == '2'){
23. dl.MainMenu();
24. }
25. else if ( c == '3'){
26. exit(0);
27. }
28. }
29. };
30. int main()
31. {
32. MainLinkList mll;
33. mll.MainLinkListModule();
34. cout<<"\n\nThank You for Using My App..\m\m";
35. cout<<"Design & Coded By – Tarun Manghani & Bharat Soni \n\n";
36. }
```



Dezyne École College
www.dezyneecole.com

Thank You